

# Volatility-Adjusted Memory Protection

A small continual learning experiment · Richard · 2026

**TL;DR** — Standard continual learning methods (EWC) protect all past knowledge with the same fixed strength, forcing a tradeoff between remembering old tasks and adapting to new ones. I tested whether scaling protection by how *volatile* each task's domain actually is — measured before any protection is applied — can improve *both* sides simultaneously. On a synthetic benchmark it did: **+2.7pp stable retention and +5.2pp volatile adaptation** over a uniformly-tuned baseline.

## 1. The Problem

Continual learning systems face the **stability-plasticity dilemma**: protect old knowledge too strongly and the model cannot adapt; too weakly and it forgets. Elastic Weight Consolidation (EWC) penalises changes to parameters proportional to their past importance (Fisher information):

$$\mathcal{L}_{EWC} = \mathcal{L}_{CE} + \lambda \sum_i F_i (\theta_i - \theta_i^*)^2$$

EWC works, but applies *one global protection strength uniformly* — every past task is guarded equally regardless of whether its domain is still relevant. Not all knowledge ages the same way: core language structure barely shifts; a user's current preferences might change monthly. Treating both equally wastes protection budget on memories going stale and starves memories that actually deserve to last.

## 2. Hypothesis

Scale the EWC penalty per stored task by how volatile that task's domain actually is, measured *before* any protection is applied. High measured volatility → weak protection. Low volatility → strong protection. The prediction: improve *both* retention and adaptability simultaneously — not just shift the tradeoff — by stopping budget being wasted on stale memories.

## 3. Method

### Benchmark

12 sequential binary classification tasks on 2D Gaussian blobs, alternating between two domain types sharing a *single trunk and single output head* — genuine weight competition, no dedicated capacity to hide in.

- **Stable domain:** blob centres jitter  $\pm 0.15$  — boundary barely moves.
- **Volatile domain:** blob centres relocate substantially each task.

### Volatility Estimation (pre-update)

Before any gradient step on a new task, measure the model's loss using only knowledge from prior tasks — a clean mismatch signal uncorrupted by the EWC penalty. Track a running EMA per domain:

$$V_d^{(t)} = \beta V_d^{(t-1)} + (1 - \beta) \mathcal{L}_{\text{pre}}^{(t)}$$

### Volatility-Scaled EWC Penalty

Each stored task's penalty term is now weighted by its domain's volatility estimate:

$$\mathcal{L}_{\text{vol}} = \mathcal{L}_{\text{CE}} + \lambda \sum_{\text{task}} w_d \sum_i F_i(\theta_i - \theta_i^*)^2, \quad w_d = \frac{1}{V_d^\gamma}$$

High  $V_d \rightarrow$  small  $w_d \rightarrow$  weak protection. Low  $V_d \rightarrow$  large  $w_d \rightarrow$  strong protection.

## 4. Failures Worth Keeping

Two earlier attempts produced flat ties — not because the idea was wrong, but because the experiments were too easy:

- **Separate output heads** — no genuine weight competition, nothing to forget in either method.
- **Volatility measured post-update** — contaminated by EWC clamping, tracking the intervention not the domain.
- **Models trained to 100% confidence** — collapses Fisher information toward zero, making the EWC penalty inert in both methods. Fixed by label smoothing and fewer training epochs.

Diagnosing *why* the null result occurred — rather than just re-running with different seeds — was most of the actual engineering work.

## 5. Results

Averaged over 8 runs, 12 tasks each:

| Metric                        | Baseline EWC | Volatility-Adjusted   |
|-------------------------------|--------------|-----------------------|
| Early-task stable retention   | 0.935        | <b>0.962 (+2.7pp)</b> |
| Late-task volatile adaptation | 0.576        | <b>0.628 (+5.2pp)</b> |
| Avg accuracy — stable tasks   | 0.936        | <b>0.963 (+2.7pp)</b> |
| Avg accuracy — volatile tasks | 0.617        | <b>0.633 (+1.6pp)</b> |

Volatility-adjustment improved both axes simultaneously. The learned volatility estimates also made sense independently: the model inferred the volatile domain was  $\sim 2.4\times$  more unstable than the stable domain, purely from pre-update loss — without being told the domain labels had any meaning about volatility.

## 6. Caveats

**What this is not:**

- **Toy benchmark only.** 2D blobs, 4-layer MLP. Split-MNIST/CIFAR with genuine domain drift is the natural next step.
- **One hyperparameter regime.** The effect only appeared once  $\lambda$  was high enough to create a real tradeoff.
- **No comparison to replay-based methods** or Synaptic Intelligence — volatility-weighting may be additive or redundant.

## 7. Why This Is on My Portfolio

This started as a philosophical question — when should a person trust an old habit vs. question it — that turned out to have a direct mechanistic analogue in a real ML problem. The path from idea to equations to working code to null result to diagnosed bug to real result is most of what applied ML engineering is day-to-day.

Being a self-taught engineer without a formal ML background, and arriving at a result that maps onto a genuine open problem in continual learning research, is the actual thing this piece demonstrates.

---

Code available on request · richard@[domain]